

Extensions and themes

Themes for Google Chrome are quite similar to extensions, and in fact come in the same CRX format

Chrome addon development

Chrome extensions page on Google Code is a good place to start if you want to develop extensions <http://code.google.com/chrome/extensions/>

Developer corner

This HTML file will contain the code for changing and saving the settings used in the extension.

The `browser_action` field defines that this is a browser action plug-in, i.e. one which will appear on the toolbar and will be visible in the browser at all times. If you want the icon to only appear in certain situations you can use a page action instead.

You can only use one of page action

and browser action in an extension.

Under `browser_action`, we have `default_icon`, and `popup`. The default icon, as you might be able to guess, is set to the location of the icon image which will be shown on the toolbar on the browser. The `popup` field is set to the location to the file which will be used as a popup when someone clicks on the icon.

The `permissions` field is used to declare the web sites from where you

extension will load data. For security sake, Chrome ensures that you only get access to these web sites. Here we are going to be accessing the RSS feed of the thinkdigit web site, and as such we tell Chrome to permit the extension to load resources from there. However, you can also set this field to `http://*/` to gain access to any web site!

The `default_icon` can be any image (JPG, PNG, GIF) which should optimally

EXTENDING THE EXAMPLE TO USE ANY RSS FEED

We can quite easily make this extensions work for any RSS feed instead of just one, and can in fact leave the option up to the user. Let us see how we can go about doing that.

We can use `localStorage` to store the URL of the feed, and load this URL instead of loading a preset one. Just to make sure that there is always a feed to load, we can have a preset value pointing to a feed. Or you can add logic which prompts the user to configure the extension if the URL hasn't been set. Since asking the user is better, let us do that.

First we check if the URL has been stored in the `localStorage`, and if it hasn't, display a link to the options page instead of the links from the feed. To do that we need to make a few changes though. Since you cannot add anything to the popup till it loads, we will shift all our code to a new `main` function, and call it when the popup is done loading as follows:

```
var req = new XMLHttpRequest();
function main()
{
    if(window.
localStorage["feedURL"])
    {
        req.open("GET",window.
localStorage["feedURL"], true);
        req.onload = showTDRssFeed;
        req.send(null);
    } else {
```

```
var optionsLink = document.
createElement("a");
optionsLink.href = chrome.
extension.getURL("options.html");
optionsLink.innerText =
"Options page";
document.body.innerHTML =
"Before you begin using the
extension, you need to configure
it from its ";
document.body.
appendChild(optionsLink);
}
```

Since earlier all additions to the document were being done after the feed loaded (which is bound to be after the popup loads, there was no problem.

Now we are checking if a `localStorage` property called `feedURL` has been defined, and if it doesn't, we are adding a message telling the user to visit the options page before proceeding. We can get a link to the options page using `chrome api` `chrome.extension.getURL("options.html")`.

Now if we don't have a `feedURL` set, it will prompt you to open the extension options page. You could set the extension to automatically open that page if no URL is found if you want. We will also need to add the `body` tag to the same `popup.html` file with the `main` function set to run once it is done loading:

```
<body onload="main()">
</body>
```

We will also need to add the option parameter in the "options.html" page. You can simply add the following after the first option block:

```
<label for="feedURL" >What RSS
feed do you wish to load stories
from?</label>
<input id="feedURL" type="text"
/>
<input type="button"
onclick="storeURL()" value="Set"
/>
```

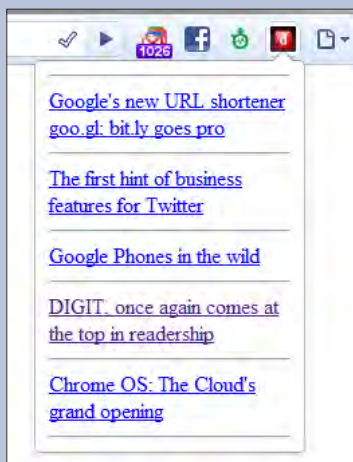
And the `storeURL` function is simply:

```
function storeURL()
{
    window.
localStorage["feedURL"] = docu-
ment.getElementById("feedURL").
}
```

Finally, to make sure that your extension has permission to load any feed it wants, the `permissions` block should become:

```
"permissions": [
    "http://*/"
]
Instead of:
"permissions": [
    "http://www.thinkdigit.
com/"
]
```

There you have it! A simple Chrome extension which can display items from any feed in a dropdown! Go ahead, start building on this, and make an extension which can load from multiple feeds, or one which allows paging. There are many possibilities even in this small example.



Your very own Chrome extension at work